A Utility for Creating Execution Environments

Document #:	P3325R0
Date:	2024-06-13
Project:	Programming Language C++
Audience:	LEWG Library Evolution

Reply-to:	Eric Niebler <eric.niebler@gmail.com>

Contents

"The environment is everything that isn't me."
— Albert Einstein

§ 1 Introduction

Execution environments are a fundamental part of the `std::execution` proposal, [P2300R9], but that paper provides no utility for creating or manipulating environments.

Such utilities are increasingly necessary considering that some proposed APIs (e.g., `write_env` from [P3284R0]) require passing an environment.

This paper proposes two simple utilities for creating execution environments: one to create an environment out of a query/value pair and another to join several environments into one.

§ 2 Executive Summary

This paper proposes the following changes to P2300R9:

— Add a class template provisionally called `prop` to the `std::execution` namespace. `prop` associates a query `Q` with a value `V`, yielding a *`queryable`* object `E` such that `E.query(Q)` is equal to `V`.

— Add a class template provisionally called `env` to the `std::execution` namespace. `env` aggregates several environments into one, giving precedence to the environments in lexical order.

— Optionally, replace `empty_env` with `env<>`.

§ 3 Discussion

§ 3.1 Motivation

In [P2300R9], execution environments are an internal implementation detail of the sender algorithms. There are no public-facing APIs that accept environment objects as a parameter. One may wonder why a utility for constructing environments is even necessary.

There are several reasons why the Standard Library would benefit from a utility to construct an environment:

1. The set of sender algorithms is openly extensible. For those who decide to implement their own sender algorithms, the manipulation of execution environments is part of the job. Standard utilities will make this easier.

2. [P3284R0] proposes a `write_env` sender adaptor that merges a user-specified environment with the environment of the receiver to which the sender is eventually connected. Using this adaptor requires the user to construct an environment. Although implementing an environment is not hard, an out-of-the-box solution will make using `write_env` simpler.

3. [P3149R3] proposes two new algorithms, `spawn` and `spawn_future`, both of which can be parameterized by passing an environment as an optional argument.

4. Currently, there is no way to parameterize algorithms like `sync_wait` and `start_detached`. The author intends to bring a paper proposing overloads of those

functions that accept environments as a way to inject things like stop tokens, allocators, and schedulers into the asynchronous operations that those algorithms launch.

In short, environments are how users will inject dependencies into async computations. We can expect to see more APIs that will require (or will optionally allow) the user to pass an environment. Thus, a standard utility for making environments is desirable.

§ 3.2 Design Considerations

§ 3.2.1 Singleton Environments

Now that `tag_invoke` has been removed from P2300, defining an execution environment is quite simple. To build an environment with a single query/value pair, the following suffices:

```
template <class Query, class Value>
struct prop
{
  [[no_unique_address]] Query query_;
  Value value_;

  auto query(Query) const noexcept -> const Value &
  {
    return value_;
  }
};

// Example usage:
constexpr auto my_env = prop(get_allocator, std::allocator{});
```

Although simple, this template has some nice properties:

1. It is an aggregate so that members can be direct initialized from temporaries without so much as a move. A function that constructs and returns an environment using `prop` will benefit from RVO:

   ```
   // The frobnicator object will be constructed directly in
   // the callers stack frame:
   return prop(get_frobnicator, make_a_frobnicator());
   ```

2. An object constructed like `prop(get_allocator, my_allocator{})` will have the simple and unsurprising type `prop<get_allocator_t, my_allocator>`.

The `prop` utility proposed in this paper is only slightly more elaborate than the `prop` class template shown above, and it shares these properties.

§ 3.2.2 Environment Composition

What if you need to construct an environment with more than one query/value pair, say, an allocator and a scheduler? A utility to join multiple environments would work together with prop to make a general environment-building utility. This paper calls that utility env.

The following code demonstrates:

```
template <class Env, class Query>
concept has-query = requires (const Env& env) { env.query(Query()); };

template <class... Envs>
struct env : Envs...
{
  template <class Query>
  static constexpr size_t _index_of() {
    constexpr bool flags[] = {has-query<Envs, Query>...};
    return ranges::find(flags, true) - flags;
  }

  template <class Query>
    requires (has-query<Envs, Query> ||...)
  decltype(auto) query(Query q) const noexcept(...)
  {
    auto tup = tie(static_cast<const Envs&>(*this)...);
    return get<_index_of<Query>()>(tup).query(q);
  }
};

template <class... Envs>
env(Envs...) -> env<Envs...>;

// Example usage:
constexpr auto my_env = env(prop(get_allocator, my_alloc{}),
                            prop(get_scheduler, my_sched{}));
```

This env class template shares the desirable properties of prop: aggregate initialization and unsprising naming. A query is handled by the “leftmost” child environment that can handle it.

Note that the above code has the issue that two child environments cannot have the same type. That is not a limitation of the facility proposed below.

§ 3.2.3 By Value vs. By Reference

There are times when you would like an environment to respond to a query with a reference to a particular object rather than a copy. Capturing a reference is dangerous, so the opt-in to reference semantics should be explicit.

std::reference_wrapper is how the Standard Library deals with such problems. It should be possible to construct an environment using std::ref to specify that the “value” should be stored by reference:

```
std::mutex mtx;
const auto my_env = prop(get_mutex, std::ref(mtx));
std::mutex & ref = get_mutex(my_env);
assert(&ref == &mtx);
```

Similarly, there are times when you would like to store the child environment by reference. `std::ref` should work for that case as well:

```
// At namespace scope, construct a reusable environment:
constexpr auto global_env = prop(get_frobnicator, make_frobnicator());

// Construct an environment with a particular value for the get_allocator
// query, and a reference to the constexpr global_env:
auto env_with_allocator(auto alloc)
{
  return env(prop(get_allocator, alloc), std::ref(global_env));
}
```

The utility proposed in this paper satisfies all of the above use cases.

§ 3.3 Naming

For the `prop` utility, a couple of other plausible names come to mind:

— `attr`

— `with`

— `property`

— `key_value`

— `query_value`

Other possibile names for `env` are:

— `attributes`, or `attrs`

— `properties`, or `props`

— `dictionary`, or `dict`

§ 4 Implementation Experience

Types with a very similar designs have seen heavy use in `stdexec` for over two years. The design proposed below has been prototyped and can be found on Compiler Explorer[1].

A note on implementability: The implementation of `env` is simple if we only want to support braced list initialization with class template argument deduction, like `env{e1, e2, e2}`. Initialization from a parenthesized expression list with CTAD for an arbitrary number of arguments (e.g., `env(e1, e2, e3, ...)`) is not implementable in standard C++ to the author's knowledge.

In the proposed wording, `env` is specified such that it will be able to benefit from member packs ([P3115R0]) if and when they become available or should aggregate initialization with a parenthesized expression list ever be extended to support brace elision for subobjects.

§ 5 Future Extensions

There are several ways this utility might be extended in the future:

- Add a way to *remove* a query from an environment.
- Add a wrapper for a reference to an environment.
- Add a wrapper that only accepts queries that are *forwarding* (see [exec.fwd.env]).

§ 6 Proposed Wording

[Editor's note: Replace all occurances of `empty_env` with `env<>`. Change [exec.syn] as follows:]

Header `<execution>` synopsis [exec.syn]

```
namespace std::execution {
  …as before…

  struct empty_env {};
  struct get_env_t { see below };
  inline constexpr get_env_t get_env{};

  template<class T>
    using env_of_t = decltype(get_env(declval<T>()));

  // [exec.prop] class template prop
  template<class Query, class Value>
    struct prop;

  // [exec.env] class template env
  template<queryable... Envs>
    struct env;

  // [exec.domain.default], execution domains
  struct default_domain;

  // [exec.sched], schedulers
  struct scheduler_t {};

  …as before…
}
```

[Editor's note: After [exec.utils], add a new subsection [exec.envs] as follows:]

34.12 Queryable utilities [exec.envs]

34.12.1 Class template `prop` [exec.prop]

```
namespace std::execution {
  template<class Query, class Value>
  struct prop {
```

```
    [[no_unique_address]] Query query;    // exposition only
    [[no_unique_address]] Value value;    // exposition only

      [[nodiscard]] constexpr const Value& query(Query) const
        noexcept {
      return value;
    }
  };

  template<class Query, class Value>
  prop(Query, Value) -> prop<Query, unwrap_reference_t<Value>>;
}
```

1. Class template `prop` is for building a queryable object from a query object and a value.

2. [*Example 1:*

```
template<sender Sndr>
sender auto parameterize_work(Sndr sndr) {
  // Make an environment such that get_allocator(env)
        returns a reference to a copy
  // of my_alloc{}
  auto e = prop(get_allocator, my_alloc{});

   // parameterize the input sender so that it will
        use our custom execution environment
  return write_env(sndr, e);
}
```

— *end example*]

34.12.2 Class template env [exec.env]

```
namespace std::execution {
  template<class Env, class Query, class... Args>
    concept has-query =                   // exposition only
      requires (const Env& env, Args&&... args) {
        env.query(Query(), std::forward<Args>(args)...);
      };

  template<queryable... Envs>
  struct env {
    Envs_0 envs_0;       // exposition only
    Envs_1 envs_1;       // exposition only
      ...
    Envs_n-1 envs_n-1;   // exposition only

    template<class Query, class... Args>
     constexpr decltype(auto) get-first() const noexcept {   //
        exposition only
           constexpr bool flags[] = {has-query<Envs, Query,
        Args...>..., false};
      constexpr size_t idx = ranges::find(flags, true) - flags;
      return (envs_idx);
```

```
  }

  template<class Query, class... Args>
    requires (has-query<Env, Query, Args...> ||...)
      [[nodiscard]] constexpr decltype(auto) query(Query q,
      Args&&... args) const
        noexcept(noexcept(get-first<Query, Args...>().query(q,
      std::forward<Args>(args)...))) {
              return get-first<Query, Args...>().query(q,
      std::forward<Args>(args)...);
  }
};

template<class... Envs>
  env(Envs...) -> env<unwrap_reference_t<Envs>...>;
}
```

1. The class template `env` is used to construct a queryable object from several queryable objects. Query invocations on the resulting object are resolved by attempting to query each subobject in lexical order.

2. It is unspecified whether `env` supports class template argument deduction ([over.match.class.deduct]) when performing initialization using a parenthesized *expression-list* ([dcl.init]).

3. [*Example 1:*

```
template<sender Sndr>
sender auto parameterize_work(Sndr sndr) {
  // Make an environment such that:
  //   - get_allocator(env) returns a reference to a
       copy of my_alloc{}
  //   - get_scheduler(env) returns a reference to a
       copy of my_sched{}
  auto e = env{prop(get_allocator, my_alloc{}),
               prop(get_scheduler, my_sched{})};

  // parameterize the input sender so that it will
       use our custom execution environment
  return write_env(sndr, e);
}
```

— *end example*]

§ 7 Acknowledgements

I would like to thank Lewis Baker for many enlightening conversations about the design of execution environments. The design presented here owes much to Lewis's insights.

§ 8 References

[P2300R9] Eric Niebler, Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Bryce Adelstein Lelbach. 2024-04-02. `std::execution`.
https://wg21.link/p2300r9

[P3115R0] Corentin Jabot. 2024-02-15. Data Member, Variable and Alias Declarations Can Introduce A Pack.
https://wg21.link/p3115r0

[P3149R3] Ian Petersen, Ján Ondrušek; Jessica Wong; Kirk Shoop; Lee Howes; Lucian Radu Teodorescu; 2024-05-22. async_scope — Creating scopes for non-sequential concurrency.
https://wg21.link/p3149r3

[P3284R0] Eric Niebler. 2024-05-16. `finally`, `write_env`, and `unstoppable` Sender Adaptors.
https://wg21.link/p3284r0

---

1. https://godbolt.org/z/976b1G45a↩